Department of Electronics & Communications

25SC2107E – Machine Learning

Skilling Experiment 1

Titanic Survival

Dataset: Titanic | Course Outcome: CO1 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Aziz Nagar

1 Experiment : Titanic Survival

Dataset: Titanic **Course Outcome:** CO1

1.1 Aim

To set up the machine-learning workspace, load the Titanic dataset, perform exploratory data analysis, map each artifact of the analysis onto a stage of the machine-learning lifecycle, and export a multi-page EDA report as a PDF.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Load a dataset and produce a structured summary of its shape, dtypes and missingness.
- Visualise distributions, group-wise rates and correlations.
- Export a multi-page PDF report using `PdfPages`.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib seaborn scikit-learn
```

1.4 Procedure

1. Create a notebook `skill_lab1.ipynb` in the workspace;
2. Import the libraries and load the Titanic dataset.
3. Print the structure: `info()`, `describe()` and the class balance.
4. Count the missing values, then view them as a heatmap.
5. Plot survival rate by sex, then by class, then the age distribution split by survival.
6. Compute and plot the numeric correlation heatmap.
7. Export a PDF report to `titanic_eda_report.pdf` using `PdfPages`.

1.5 Notebook Implementation

Cell 1 — Import the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.backends.backend_pdf import PdfPages
```

Explanation: Five imports, and each has one job. `pandas` holds the table, `numpy` does the arithmetic underneath it, `matplotlib` draws, `seaborn` draws the same thing with less typing, and `PdfPages` collects several figures into one PDF file at the end. Run the cell with `Shift+Enter`. If it produces no output, everything is installed correctly.

1.6 Dataset and Link

Titanic — 891 passengers; features include class, sex, age, fare, port of embarkation; target is `survived`.

Where the data actually comes from. `sns.load_dataset("titanic")` is *not* a copy of the dataset bundled inside the seaborn package. The function builds a URL and downloads a CSV over

the internet, from a small GitHub repository the seaborn author maintains for its documentation examples:

https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv

So the third link below is the true source — `load_dataset` is just a one-line wrapper around `pd.read_csv` of that address. Two consequences follow: the cell needs a working internet connection, and if the repository is unreachable (offline lab, firewall, proxy) you must fall back to `pd.read_csv` with a local copy of the file.

- Convenience loader: `seaborn.load_dataset("titanic")` — downloads the CSV below
- Actual file fetched: https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv
- Original source: https://www.kaggle.com/c/titanic/data

Note

If you don't have internet. Download `titanic.csv` once from the raw link above, place it beside your notebook, and replace Cell 2's first line with:

```
titanic = pd.read_csv("titanic.csv")
```

Cell 2 — Load the dataset

```
titanic = sns.load_dataset("titanic")
print(titanic.shape)
titanic.head()
```

Explanation: `load_dataset` downloads the Titanic CSV once and returns a DataFrame, so there is no file to manage. `shape` prints `(891, 15)` — 891 passengers, 15 columns. `head()` shows the first five rows; place it on the last line of the cell and Jupyter renders it as a proper table.

Cell 3 — Structure of the data

```
titanic.info()
titanic.describe()
titanic["survived"].value_counts(normalize=True)
```

Explanation: `info()` lists every column with its dtype and its non-null count; wherever that count is below 891 you have missing data. `describe()` gives the numeric summary. `value_counts(normalize=True)` gives the class balance, roughly 62% died and 38% survived.

Cell 4 — Count the missing values

```
titanic.isnull().sum().sort_values(ascending=False)
```

Explanation: Read this left to right. `isnull()` turns the table into True/False, `sum()` counts the Trues in each column (True counts as 1), and `sort_values` puts the worst offenders on top. You will find `deck` missing 688 times

Cell 5 — Where the values are missing

```
sns.heatmap(titanic.isnull(), cbar=False, cmap="Reds")
plt.title("Missing Values")
plt.show()
```

Explanation: The previous cell said *how many* are missing; this one says *where*. Each row of the image is a passenger and each red mark is an absent cell. The `deck` column is almost solid red, while `age` is speckled. Look closely at the pattern: cabin data is missing mostly for lower-class passengers, so the missingness itself carries information.

Cell 6 — Survival rate by sex

```
sns.barplot(data=titanic, x="sex", y="survived")
plt.title("Survival Rate by Sex")
plt.show()
```

Explanation: Because `survived` is 0 or 1, the mean of the column *is* the survival rate — and `barplot` plots the mean by default. So the bar height reads directly as a probability: about 0.74 for women against 0.19 for men. The thin line on each bar is a 95% confidence interval, seaborn's way of saying how much it trusts the height.

Cell 7 — Survival rate by class

```
sns.barplot(data=titanic, x="pclass", y="survived")
plt.title("Survival Rate by Class")
plt.show()
```

Explanation: The same one-line recipe with a different `x`. Survival falls steadily from first class (≈0.63) to third (≈0.24). Note that `pclass` is stored as the number 1, 2, 3 but behaves as a *ranked category*;

Cell 8 — Age distribution split by survival

```
sns.histplot(data=titanic, x="age", hue="survived", kde=True)
plt.title("Age vs Survival")
plt.show()
```

Explanation: A histogram buckets ages and counts passengers per bucket; `hue="survived"` draws one coloured histogram per outcome, and `kde=True` overlays a smooth curve. The two curves overlap heavily — age is not a strong separator overall — but the left edge tells a different story: below about ten years, survivors clearly outnumber non-survivors. That local effect is why the "women and children first" order shows up in the data.

Understanding the Correlation Heatmap

Correlation Heatmap — The Simple Version

One question, asked many times. The heatmap asks a single question about every pair of number columns: *when one goes up, what does the other do?* There are only three possible answers.

Value	Colour	Meaning
near +1	strong red	They rise **together**.
near 0	pale / white	**No straight-line link.**
near −1	strong blue	One rises, the other **falls**.

Applied to the Titanic squares:

- `fare` and `survived`, +0.26 — "When the ticket price goes up, survival tends to go **up**." *Richer passengers survived more often.*
- `pclass` and `survived`, −0.34 — "When the class number goes up, survival tends to go **down**." *And remember class 3 is the cheapest, so this says the same thing again: better class, better odds.*
- `sibsp` and `parch`, +0.41 — "When siblings aboard goes up, parents/children aboard tends to go **up**." *People travelled as families.*

Cell 9 — Correlation heatmap

```
cols = ["survived", "pclass", "age", "sibsp", "parch", "fare"]
cols = [c.strip() for c in cols]
sns.heatmap(titanic[cols].corr(), annot=True, fmt=".2f",
    cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

1.7 Expected Output

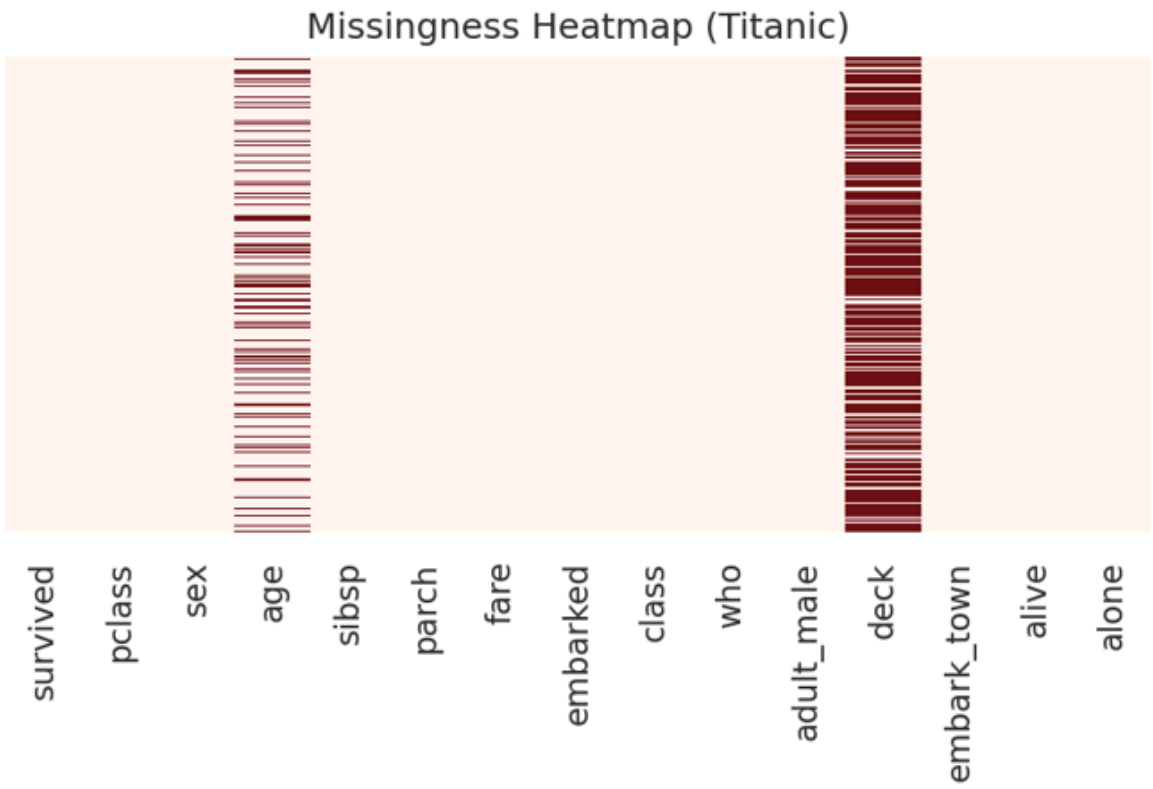

Figure 1: Missingness heatmap and per-column missing counts. `deck` is unusable; `age` is repairable.

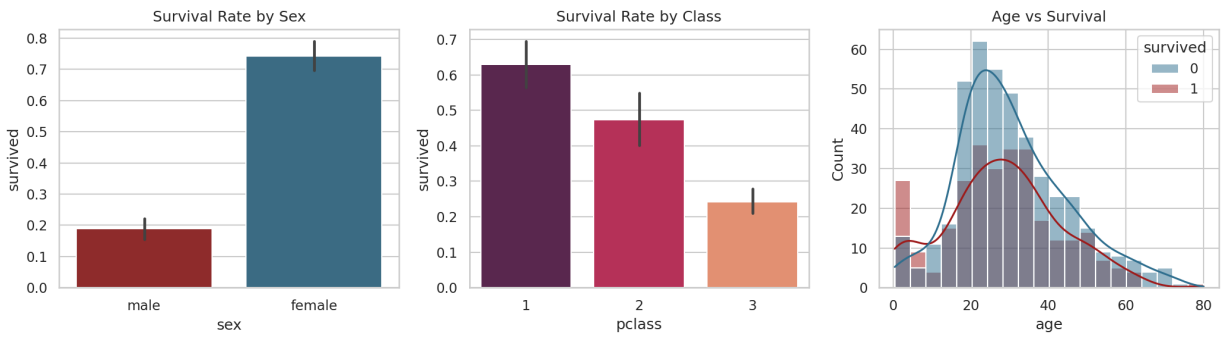

Figure 2: Survival rate by sex and class, and the age distribution split by outcome. The error bars are bootstrapped 95% confidence intervals.

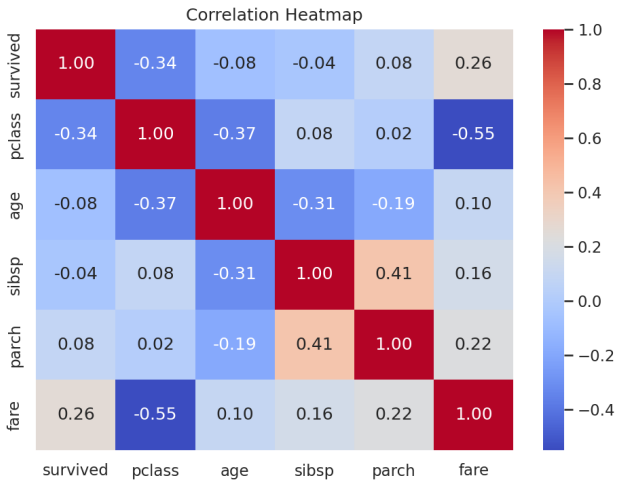

Figure 3: Correlation of numeric features with survival.

1.8 Result

The VS Code workspace was configured, the Titanic dataset was profiled for structure and missingness, survival patterns were visualised and quantified and EDA report was exported to `titanic_eda_report.pdf`.

1.9 What is an EDA Report?

EDA stands for *Exploratory Data Analysis* — the stage in which you interrogate a dataset before you model it. You are trying to answer four questions:

- what is in this data?
- what is missing from it?
- what patterns does it already contain?
- what will break if I feed it to an algorithm as it stands.

An **EDA report** is the frozen, shareable *record* of that interrogation. This distinction matters more than it first appears:

Deliverable Function

```python
def titanic_eda_report(df, path="titanic_eda_report.pdf"):
    pdf = PdfPages(path)

    # Page 1 - missing values
    plt.figure(figsize=(11, 6))
    sns.heatmap(df.isnull(), cbar=False, cmap="Reds")
    plt.title("Page 1 - Missing Values")
    pdf.savefig()
    plt.close()

    # Page 2 - survival by sex
    plt.figure(figsize=(11, 6))
    sns.barplot(data=df, x="sex", y="survived")
    plt.title("Page 2 - Survival Rate by Sex")
    pdf.savefig()
    plt.close()

    # Page 3 - correlation
    cols = ["survived", "pclass", "age", "sibsp", "parch", "fare"]
```

```
    plt.figure(figsize=(11, 6))
    sns.heatmap(df[cols].corr(), annot=True, fmt=".2f",
        cmap="coolwarm")
    plt.title("Page 3 - Correlation Heatmap")
    pdf.savefig()
    plt.close()


    pdf.close()
    print("Report written to", path)


titanic_eda_report(titanic)
```

1.10 TODO — Modify and Extend the Code

TODO — Tinker with the Code

Group A — Missing values (edit Cells 4 and 5)

1. **Percentages instead of counts.** Cell 4 tells you `deck` is missing 688 times. That number only means something if you also remember the dataset has 891 rows. Rewrite the cell to print the missing *percentage* of each column, rounded to one decimal, showing only the columns that actually have gaps.
   Hint: divide by `len(titanic)`, then filter with `pct[pct > 0]`.
2. **A less crowded heatmap.** Cell 5 draws all 15 columns, but 11 of them are completely full and contribute nothing but white space. Redraw the heatmap using only the columns that contain at least one missing value.
   Hint: `titanic.columns[titanic.isnull().any()]` gives you exactly those names.

TODO — Tinker with the Code

Group B — Survival plots (edit Cells 6, 7 and 8)

3. **Add a reference line.** A bar at 0.74 means little until you know the overall survival rate was 0.38. Add a dashed horizontal line at the overall rate to the survival-by-sex chart, so every bar can be read as above or below average.
   Hint: `plt.axhline(titanic["survived"].mean(), color="black", linestyle="--")`.
4. **Two variables in one chart.** Cells 6 and 7 look at sex and class separately. Show both at once by adding `hue="sex"` to the class chart. Then answer a question neither original chart could: *did third-class women fare better or worse than first-class men?*

TODO — Tinker with the Code

Group C — Correlation (edit Cell 9)

5. **Build an entirely positive and an entirely negative map.** Draw two heatmaps of the same correlation matrix: one showing *only* the positive values, one showing *only* the negative. Cells that fail the test should be left blank rather than coloured.
   Hint: `corr[corr > 0]` keeps the positive entries and replaces the rest with `NaN`, which seaborn draws as empty. Set `vmin=0` on the positive map and `vmax=0` on the negative one so each colour scale starts at zero.

TODO — Tinker with the Code

Group D — The EDA report (edit the Deliverable)

6. **Add a findings page.** Your report shows three pages of plots and states no conclusions — and, as the section *What is an EDA Report?* argued, a report that draws no conclusion is a

slideshow. Add a fourth page carrying your own written findings.
The technique is new, so here it is in full. A matplotlib figure does not have to contain a plot: you can hide the axes and write directly onto the blank page.

```
plt.figure(figsize=(11, 6))
plt.axis("off")                                    # hide the empty
    axes
plt.text(0.02, 0.94, "Findings", size=16, weight="bold")
plt.text(0.04, 0.82, "1. Your first finding here.", size=11)
pdf.savefig()
plt.close()
```

Coordinates run from `0` (bottom/left) to `1` (top/right). Rather than writing one `plt.text` line per finding, put your sentences in a list and loop over it, moving `y` down by a fixed step each time.
Write at least five findings, and make every one *quantitative*: "women survived more" is weak, "74% of women survived against 19% of men" is a finding.

Note

What is seaborn?
Seaborn is a plotting library built *on top of* matplotlib. It does not replace matplotlib — it is a shorter way of calling it.
You pass the whole table plus the *names* of the columns to put on each axis:

```
sns.barplot(data=titanic, x="sex", y="survived")
```

Seaborn then does the grouping, averaging, colouring and axis labelling for you. In plain matplotlib the same chart needs you to split the data by sex, compute each mean, and label both axes by hand. The convention `import seaborn as sns` is universal, so every example you find online uses the `sns.` prefix.

Note

Name	Contents	Typically used to practise
`titanic`	891 passengers	classification, missing data, EDA
`iris`	150 flowers, 3 species	multi-class classification, `pairplot`
`penguins`	344 penguins, 3 species	classification; a modern replacement for iris
`tips`	244 restaurant bills	regression, categorical grouping
`diamonds`	53,940 diamonds	regression on a large table, skewed prices
`mpg`	398 cars	regression, fuel efficiency vs weight
`planets`	1,035 exoplanets	missing data, log-scaled distributions
`flights`	monthly passengers, 1949–60	time series, seasonality, heatmaps
`dowjones`	Dow Jones index	time series, long-run trend
`seaice`	Arctic sea-ice extent	time series, trend over decades
`healthexp`	spending vs life expectancy	multi-country line plots
`car_crashes`	crash stats per US state	correlation, ranked bar charts
`taxis`	NYC taxi trips	datetimes, geographic scatter
`fmri`	brain signal over time	line plots with confidence bands
`brain_networks`	fMRI correlations	wide tables, `clustermap`
`anscombe`	Anscombe's quartet	why you must plot, not only compute
`geyser`	Old Faithful eruptions	two-cluster (bimodal) distributions
`exercise`	pulse under treatment	repeated-measures designs
`attention`	attention experiment	within-subject comparisons
`anagrams`	anagram-solving scores	wide-to-long reshaping
`dots`	neural decision task	faceted line plots
`glue`	NLP model benchmark scores	heatmaps of model comparisons